Nosso Treino — Documentação do Projeto

Autor: Gui — estudante de Análise e Desenvolvimento de Sistemas
App no ar: nosso-treino.web.app
Stack: HTML5, CSS3 e JavaScript puro + Firebase (Auth, Firestore, Hosting)

1. O que é isso aqui

O Nosso Treino é um app web para casais que treinam juntos. Cada pessoa tem os próprios treinos (A até E), o próprio calendário e o próprio progresso. O casal se conecta por um ID, e um vê o treino e o calendário do outro — mas só vê, não mexe. Tem também as cutucadas ("Bora!" e "Larga o celular frango!") para um empurrar o outro.

Ele nasceu como projeto de estudo da minha graduação em ADS e virou produto real: eu e minha namorada usamos todo dia. Essa documentação explica o que a gente decidiu, por que decidiu e o que ficou aberto. Sem enrolação.

2. Como começou

O problema era simples: a gente treinava junto, mas cada um acompanhava do seu jeito, em app diferente. Os apps de mercado são competitivos (ranking, leaderboard), focados no indivíduo ou pagos. Nenhum servia para um casal que quer se apoiar, não disputar.

Aí eu decidi construir o app que a gente queria usar. Desenvolvi em etapas no VS Code, testando no Live Server e publicando no Firebase Hosting. Minha namorada foi a tester oficial de todas as versões.

3. As decisões que a gente tomou

Aqui eu vou ser bem objetivo: o que decidiu, por quê, e o custo que a gente aceitou pagar. Documentação que só mostra acerto não é documentação.

3.1 Firebase em vez de Supabase

Decisão: Firebase (Authentication + Firestore + Hosting).

Por quê:

- Eu já tenho um projeto no Supabase. Então parte da decisão foi essa mesmo: eu queria testar o Firebase de verdade, num projeto real, não em tutorial.
- Login com Google no Firebase é nativo: ativa o provedor, adiciona os domínios autorizados e funciona. No Supabase eu precisaria configurar um app OAuth externo e redirects.
- Tempo real por documento (onSnapshot) veio pronto para o caso do casal: um marca treino, o outro vê sem recarregar.
- Hosting no mesmo lugar: um comando (firebase deploy) publica o site. No Supabase eu teria que trazer Vercel ou Netlify junto.

Custo aceito: banco NoSQL de documentos é mais fraco que Postgres para consultas relacionais complexas. Se o app virar relatório pesado, o Supabase ganha nesse ponto. Meu projeto no Supabase continua vivo para outros usos.

Critério	Firebase (escolhido)	Supabase (comparação)

Login Google	Provedor nativo no console + domínios autorizados	Exige app OAuth externo e configuração de redirect
Tempo real	onSnapshot por documento, pronto para uso	Realtime existe, mas pede mais montagem para o nosso caso
Hosting do site	Incluído, deploy em 1 comando	Não é o foco; precisaria de hospedagem externa
Banco	NoSQL de documentos (Firestore)	Postgres relacional (mais forte em consultas complexas)
Segurança	Regras por documento, simples de auditar	Row Level Security, também boa, outra mentalidade
Plano grátis	Suficiente para o casal usando todo dia	Também suficiente

3.2 Firebase Hosting em vez de Vercel

Decisão: publicar no Firebase Hosting.

Por quê: a gente chegou a cogitar Vercel (e eu testei o caminho), mas cancelou. Com Firebase, login, banco e site ficam no mesmo projeto e no mesmo console. Menos peça móvel. Vercel é excelente, só que para este projeto seria um terceiro no meio sem necessidade.

Custo aceito: deploy depende do Firebase CLI instalado na máquina.

3.3 HTML, CSS e JS puros, sem framework

Decisão: nada de React ou Vue. JavaScript vanilla com módulos ES.

Por quê: zero build, zero npm install para rodar, deploy direto. E porque eu quis dominar o fundamento (DOM real, estado na mão, eventos) antes de abstrair.

Custo aceito: o main.js cresceu (cerca de 2.500 linhas) e o gerenciamento de estado é manual. Se entrar uma equipe no projeto, refatorar é o primeiro passo.

3.4 Modo de produção desde o começo

Decisão: criei o Firestore em modo de produção e escrevi as regras próprias no mesmo dia.

Por quê: modo de teste deixa uma janela em que qualquer pessoa lê os dados dos casais. Preferi o trabalho de escrever regra agora a dormir preocupado depois.

Custo aceito: regra errada bloqueia o app na hora. Aconteceu nos testes, a gente corrigiu e ficou mais forte por isso.

3.5 Regra do casal: cada um é dono do próprio treino

Decisão: o par vê treinos e calendário em modo somente leitura. Nada vira "treino do casal" automaticamente. Se um quiser usar o treino do outro, copia explicitamente para um dos seus (com confirmação quando vai sobrescrever).

Por quê: requisito de produto meu desde o início: apoiar, não fundar. Treino é individual, motivação é conjunta.

Custo aceito: dois cliques a mais para copiar. Evita sobrescrita acidental, vale a pena.

3.6 LocalStorage primeiro, Firestore depois

Decisão: as primeiras versões salvavam tudo no navegador. A migração para o Firestore só aconteceu quando a feature do casal exigiu dados em aparelhos diferentes.

Por quê: validar UX rápido sem depender de rede, e migrar quando existe motivo real, não por moda.

Custo aceito: os dados locais da fase de teste não vieram junto na migração. Era teste, tudo bem.

3.7 Cutucadas com mensagem fixa, sem chat livre

Decisão: só duas mensagens permitidas: "Bora!" e "Larga o celular frango!". Validação por whitelist no envio e mensagens em subcoleção própria.

Por quê: chat livre traria moderação, abuso e custo que não são o foco do app. Duas frases com personalidade resolvem o caso do casal e ainda viraram marca do produto.

Custo aceito: o usuário não escreve texto próprio. Pode virar evolução futura, se fizer sentido.

3.8 Foto de perfil comprimida no navegador

Decisão: a foto é recortada e comprimida no cliente (canvas, 320x320, WebP) e viaja como base64 no documento do usuário, com cache local.

Por quê: custo zero de Storage, funciona offline, e o casal vê a foto um do outro em todos os avatares.

Custo aceito: base64 dentro do documento infla cada gravação. Já está mapeado: o próximo passo é Firebase Storage com URL no documento.

3.9 Biblioteca wger com fallback local

Decisão: exercícios com imagem vêm da API aberta wger.de (dados com licença), com um catálogo local de 14 exercícios como reserva.

Por quê: imagem real facilita montar treino; o fallback garante que o app funciona sem internet ou se a API cair.

Custo aceito: alguns nomes vêm em inglês quando não há tradução.

3.10 Visual: claro com escuro cinza chumbo

Decisão: tema claro padrão e tema escuro em cinza chumbo com verde limão.

Por quê: chegou a existir um escuro esverdeado e eu rejeitei. O escuro da marca é chumbo, ponto. Identidade visual não se negocia no meio do caminho.

Custo aceito: nenhum relevante; são tokens de CSS.

3.11 Mobile-first e acessibilidade

Decisão: tudo desenhado primeiro para celular (menu inferior flutuante, modais em bottom sheet), com sidebar aparecendo só no desktop. Foco visível, aria-live, reordenação por teclado e respeito a prefers-reduced-motion.

Por quê: o casal usa na academia, no celular, com pressa e mão suada. Acessibilidade não é extra, é o público real.

Custo aceito: mais trabalho de CSS (breakpoints em 374, 390, 560 e 900px).

3.12 Sessão com relógio de tempo real

Decisão: o cronômetro da sessão calcula por diferença de tempo (Date.now()), não por contagem ingênua de setInterval.

Por quê: navegador de celular estrangula aba em segundo plano. Com delta, o tempo continua honesto mesmo com a tela apagada.

Regra extra de produto: só treino concluído naturalmente (todas as séries) marca o dia no calendário. Treino encerrado antes entra no histórico como parcial e não marca o dia. O calendário precisa ser verdadeiro, senão vira enfeite.

4. Como o app funciona hoje

- **Início:** saudação com nome e data, faixa da semana, treino de hoje, cartão do casal com cutucadas e caixa de recados, meta semanal e indicadores.
- **Treinos:** A a E com contagem de exercícios, cartão principal, adicionar, editar e reordenar exercícios (arrastar pela alça ou setas do teclado), e modo sessão com séries, descanso automático, vibração e salvamento ao final.
- **Calendário:** marcar e desmarcar dias (futuro bloqueado), legenda e resumo do mês.
- **Progresso:** cinco indicadores (incluindo tempo real de hoje e da semana), gráfico dos últimos 7 dias com duração verdadeira, recordes de carga, distribuição muscular e o comparativo semanal do casal lado a lado, sem ranking.
- **Perfil:** foto, tema, ID de casal, conectar e desconectar o par, ver treinos e calendário do par e copiar treino do par para um dos meus.

5. Dados e segurança

- **usuarios/{uid}:** dados próprios (treinos, dias treinados, histórico de sessões, cutucadas enviadas, nome, foto, e-mail).
- **usuarios/{uid}/cutucadas/{id}:** recados recebidos. Saíram do documento principal para a subcoleção justamente para a escuta em tempo real ficar barata.
- **codigos/{NT-XXXXXX}:** mapa do ID de casal para o uid, usado na conexão.
- **Regras:** cada usuário lê e grava só o próprio documento; lê o documento do par apenas se estiver conectado pelo ID; códigos são legíveis por qualquer logado (é assim que a busca funciona) mas graváveis só pelo dono.
- **Autenticação:** somente Google, restrita aos domínios autorizados (localhost e os domínios do Hosting).

6. Como rodar e publicar

1. Instalar Node.js LTS e o Firebase CLI (npm install -g firebase-tools).
2. firebase login com a conta dona do projeto.
3. firebase init hosting na pasta do projeto (pasta pública ".", sem SPA, sem sobrescrever o index.html).
4. Testar local com Live Server no VS Code.
5. Publicar com firebase deploy.

Qualquer mudança futura sobe com o mesmo comando: firebase deploy.

7. O que ficou em aberto

- Migrar foto de perfil para Firebase Storage (base64 no documento tem limite).
- Histórico limitado a 100 sessões: usuário muito ativo perde dado antigo.
- PWA instalável no celular.
- Notificação push quando o par cutuca com o app fechado.
- Linha do tempo de carga por exercício (evolução do supino ao longo dos meses).

Prefiro listar isso do que esconder. Limite conhecido é limite gerenciado.

8. O que este projeto prova (ligado à minha formação em ADS)

- **Arquitetura cliente-servidor com BaaS:** separação clara entre interface, regra de negócio e camada de dados.
- **Modelagem NoSQL:** documento por usuário, subcoleção para mensagens, tabela de códigos para busca — com justificativa de custo para cada escolha.
- **Tempo real:** assinatura de documento com cleanup correto no logout e na troca de par.
- **Processamento de mídia no cliente:** recorte e compressão de imagem com canvas antes de persistir.
- **Interação nativa:** drag-and-drop com Pointer Events, fallback de teclado e preservação de estado durante a sessão.
- **Acessibilidade:** foco visível, aria-live, rótulos, movimento reduzido.
- **Segurança:** regras de banco, whitelist de mensagens, domínios autorizados.
- **Entrega:** deploy contínuo por CLI, ambiente local fiel ao publicado.
- **Produto:** requisito virou regra de negócio (casal apoia, não compete), e teste com usuário real desde a primeira versão.

9. Fechamento

O Nosso Treino começou como uma dúvida de aula e virou um app que um casal usa de verdade. As decisões acima não foram as "melhores do mercado": foram as melhores para este problema, neste momento, com este time de uma pessoa e uma tester exigente. É assim que eu entendo engenharia de software na prática: escolher com critério, aceitar o custo e documentar os dois.